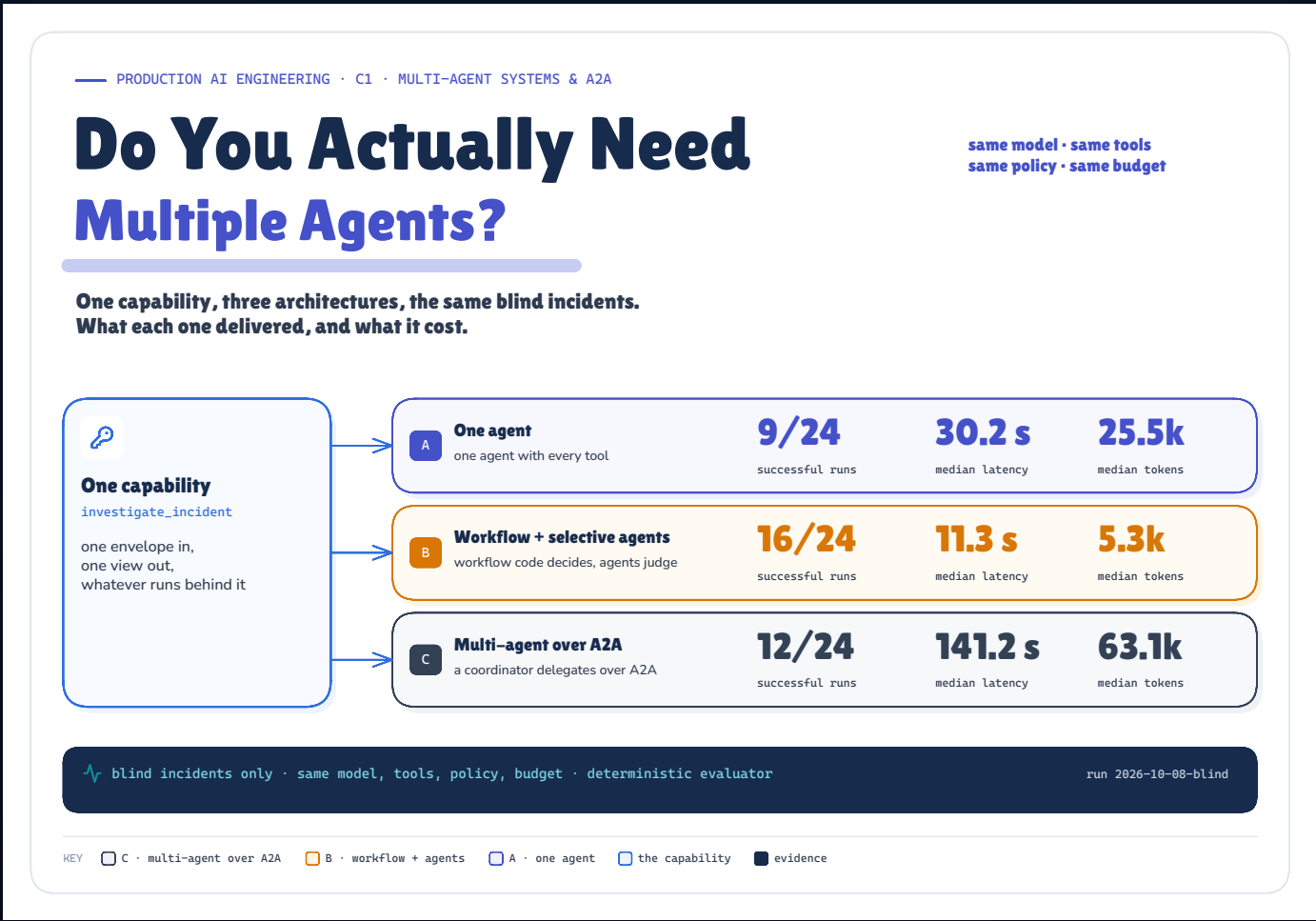


"Do You Actually Need Multiple Agents? A Controlled Experiment: One Agent, a Workflow with Agents, and Multi-Agent over A2A"

One headless capability implemented three ways (one agent, a deterministic workflow that calls agents only where judgement is needed, and a coordinator with four independent agents over A2A), run on the same eight blind incidents with the same model, tools, policy and budget. What the extra autonomy bought, what it cost, where the cost went, and when an agent boundary deserves a protocol.

C1 · Production AI Engineering
37 min read · Published 2026-10-08 · Technical deep dive



About this edition

The Medium edition makes one argument: **adding autonomous agents is an architectural decision with a measurable price, and on these eight bounded incidents, with this model, the cheapest structure that keeps control in code did best.** This edition is the reference behind it, for platform engineers and architects deciding how to structure an AI capability. It covers:

- what counts as an agent boundary, and why multiple LLM calls are not multiple agents (§2);
- three implementations of one headless capability, and exactly what was held identical (§3);
- the POC: twelve incidents with hand-written ground truth, a capability gateway, a token service whose authority only narrows, real MCP servers and real A2A agent processes (§4);
- the measured results of the blind run and where the extra cost of autonomy went (§5);
- what happens when an agent process dies mid-task, who owns workflow truth, and who owns termination (§6);
- where A2A fits, what the protocol does and does not give you, and what an A2A boundary costs on its own (§7);
- a decision framework derived from the run, and when not to use any of this (§8).

HOW TO READ THE FIGURES

- requests, the edge of the system ■ orchestration, the workflow, a human
- an agent and its model: probabilistic ■ context and retrieval ■ tools and providers' APIs
- deterministic code: gates, routing, checks ■ control plane: identity, policy, release gate ■ evidence
- enterprise systems (simulated) ■ failure, deny, conflicting state ■ executed, verified

ARCHITECTURE conceptual design **MEASURED** a recorded POC result **RECORDED** one recorded run
SIMULATED a simulated external system

Statements are marked by what they rest on:

- **Sourced.** A protocol fact or a published position, cited to a numbered source: the A2A v1.0.1 specification ^[1], its normative proto ^[2], engineering write-ups ^[12] ^[13] ^[14] and the MAST failure taxonomy ^[15].
- **Our synthesis.** A position this series takes, such as *agent memory is not workflow truth* or the agent-boundary test. No standard defines them.
- **Implemented, measured, recorded.** Behaviour of this article's POC. Numbers are substituted at build time from `coordination_poc/runs/2026-10-08-blind/facts.json`.

The POC, every recorded run and the verifier are public:

github.com/ereshzealous/ai_blogs_poc/multi_agent_a2a_poc (`make setup` && `make verify` , no model needed).

1 · Executive summary

A multi-agent design is usually drawn before it is measured. This article measures it. One capability, F3's headless `investigate_incident`, was implemented three ways and run on eight blind production incidents, three times each, with one local model (`gpt-oss:20b`), the same thirteen MCP tools, one policy, one approval rule and one token budget:

- **A · one agent** with every tool, deciding everything;
- **B · a deterministic workflow** that collects evidence by a fixed recipe and calls tool-less agents only for diagnosis, remediation planning and (conditionally) review;
- **C · a coordinator agent** that delegates over A2A to four independent agent processes (evidence, diagnosis, remediation, review), each with its own context and narrowed authority.



	A · One agent	B · Workflow + agents	C · Multi-agent over A2A
task success (of 24)	9 · 21–57%	16 · 47–82%	12 · 31–69%
complex incidents (of 9)	0	6	4
median latency	30.2 s	11.3 s	141.2 s
median tokens	25,536.5	5,253	63,108
model calls, all runs	245	44	849
runs ended by a malformed model output	0	0	4

The workflow with agents (B) had the most successes at the lowest cost. The multi-agent system (C) succeeded in 12 runs against B's 16, at 12× B's median tokens and 12.5× its median latency, and it did not do better where the hypothesis said it should (complex incidents: 4 of 9 against B's 6; H2 NOT SUPPORTED). One agent (A) handled simple incidents about as well as B (H3 SUPPORTED), but passed 0 of 9 complex runs and reached the right outcome without reading the required evidence in 10 runs. The A2A boundary cost 19 ms per delegation on average, 0.07% of C's time, over loopback on one machine: **the coordination tax dominated; in this local deployment the A2A boundary was cheap**. C's success count also rests on few incidents: incident by incident, B had more successes than C on 3 of 8, fewer on 1, and tied on 4, while C cost more than B on all 8 (\$5.1).

Make coordination deterministic. Spend intelligence where uncertainty exists.

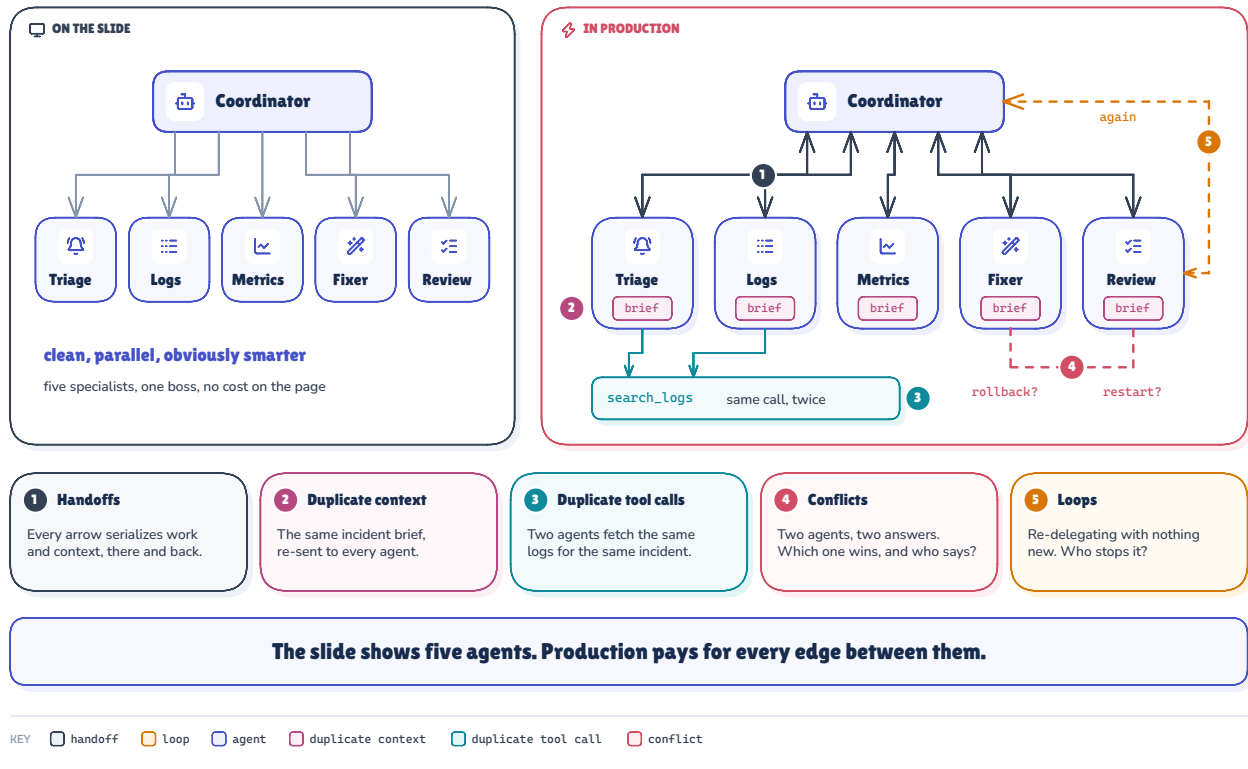
That is what this run supports, on this workload, with this model. It does not show that multi-agent systems are worse in general: published cases where they win involve breadth-first, parallelisable work across independent context windows ^[13], and nothing in these eight incidents needed that. §8 turns the evidence into a decision rule, including when multi-agent and A2A *are* the right call.

2 · The seductive diagram

Most multi-agent architectures begin as a drawing like this one: a coordinator in the middle, a triage agent, a logs agent, a metrics agent, a fixer and a reviewer around it, arrows in both directions. It looks like an organisation chart, and that is exactly why it is persuasive: we know how specialised teams outperform generalists.

A coordinator and five agents

The slide shows the boxes. Production pays for the edges between them, and those are never drawn.



ARCHITECTURE Figure 2. The usual multi-agent drawing, and the edges it leaves out: every arrow is a handoff, every box re-reads context, and every pair of agents can disagree.

Concept: the usual coordinator-and-five-agents slide and the edges it leaves out (our synthesis)

The drawing omits what every arrow costs. Each arrow is a handoff: someone must decide to make it, serialise the context it carries, wait for it, check what comes back, and decide what to do when it does not come back. Each box re-reads some of what another box already read. Each pair of boxes can reach different conclusions, and someone has to own the disagreement. None of that is reasoning about the incident. It is coordination, and in software, as in organisations, coordination is paid for.

The question this edition answers is therefore not *how to build many agents*, nor *which multi-agent framework is best*. It is narrower and more useful:

When does splitting one reasoning component into several autonomous ones produce a better production system, on the same capability, model, tools, policy and budget?

and its corollary:

When does an agent boundary deserve an interoperability protocol (A2A), as opposed to an ordinary function call?

2.1 What counts as an agent boundary

The word *agent* is used for everything from a single LLM call to an autonomous service owned by another company. For an architectural argument we need an engineering definition, not a philosophical one. Five things are routinely called agents, and they are different:

Unit	Who decides what happens next	What it owns	Example in this POC
Function	the caller's code	nothing; returns a value	<code>baseline_recipe()</code> in architecture B
Workflow step with an LLM call	the workflow's code	one judgement, structured output	B's diagnosis step
Agent	the model, within a loop	its own tool choices and turn budget, within its token's scopes	A's single agent; each of C's specialists
Agent behind a boundary	the model, inside its own process	its own context, its own tool sessions, its own failure and restart	C's specialists, each an OS process
Independent agent	its owner	its own lifecycle, release, identity and consumers	what A2A was designed for

Two equalities do **not** hold:

- *multiple LLM calls ≠ multiple agents*. Architecture B made up to 3 model calls per incident in the run and is not a multi-agent system: its code decides every step.
- *a workflow with agent steps ≠ an autonomous multi-agent system*. B delegates *judgement* to models and keeps *control* in code. C delegates control too: a coordinator model decides which agent to call next, with what, and when to stop.

An agent boundary earns its keep through properties, not through a name. The signals this edition tests are: a distinct capability; independent reasoning over its own bounded context; independently scoped authority; independent failure and retry; a different owner, runtime or release cycle; and independent consumers. None is required in every case, and having one does not make a boundary worth its cost. The run measures what the boundary costs; §8 turns that into a decision rule.

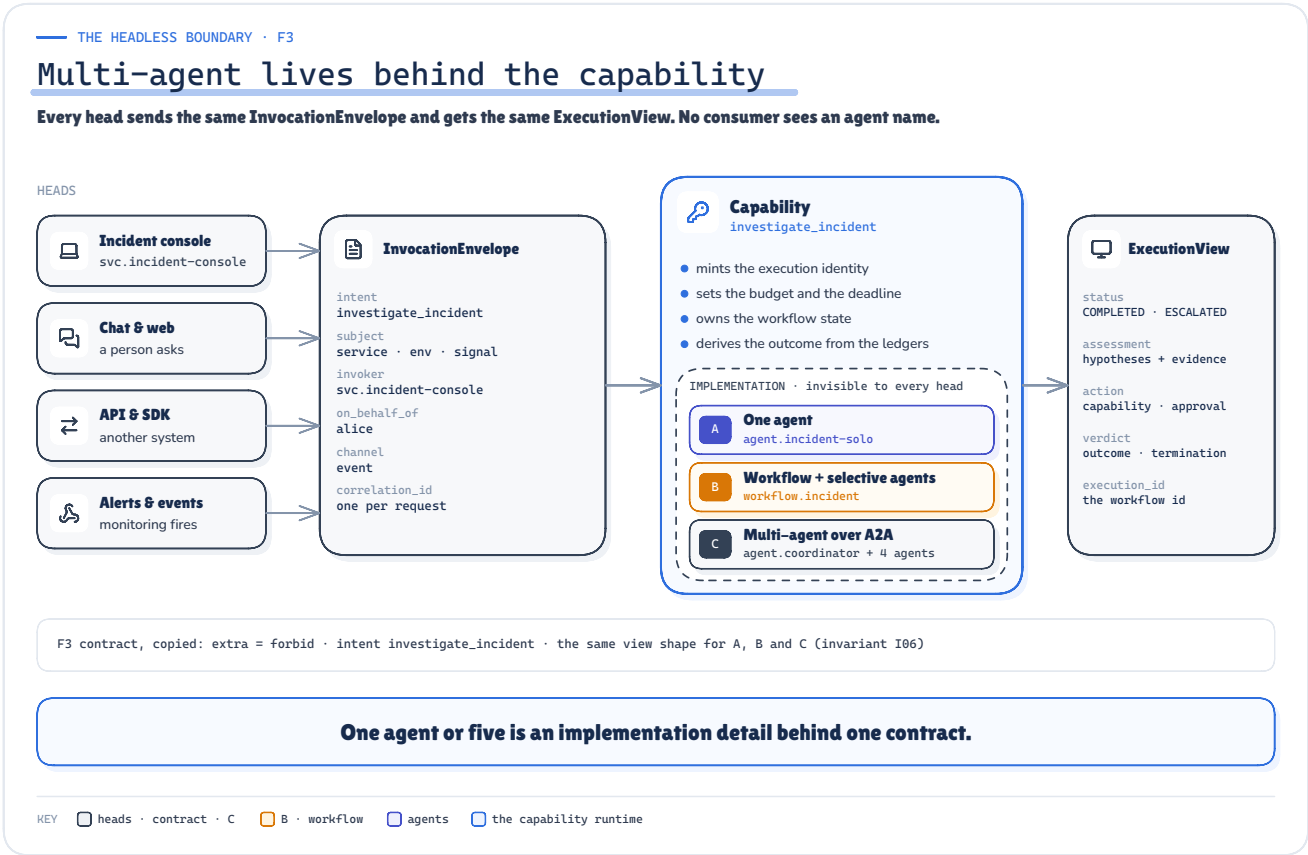
Multiple LLM calls are not multiple agents, and a workflow with agent steps is not an autonomous multi-agent system.

3 • One capability, three implementations

3.1 The headless boundary does not move

F3 established that experiences consume a *capability*, not an agent: every head (chat, web, API, event, workflow, another agent) sends an `InvocationEnvelope` and receives an `ExecutionView`, and nothing else

crosses the boundary. C1 keeps that contract byte for byte. The capability here is F3's intent `investigate_incident` ; the three architectures are three implementations behind it.



ARCHITECTURE

Figure 3. Multi-agent lives behind the capability boundary: the consumer sends the same envelope and receives the same view, whichever implementation runs.

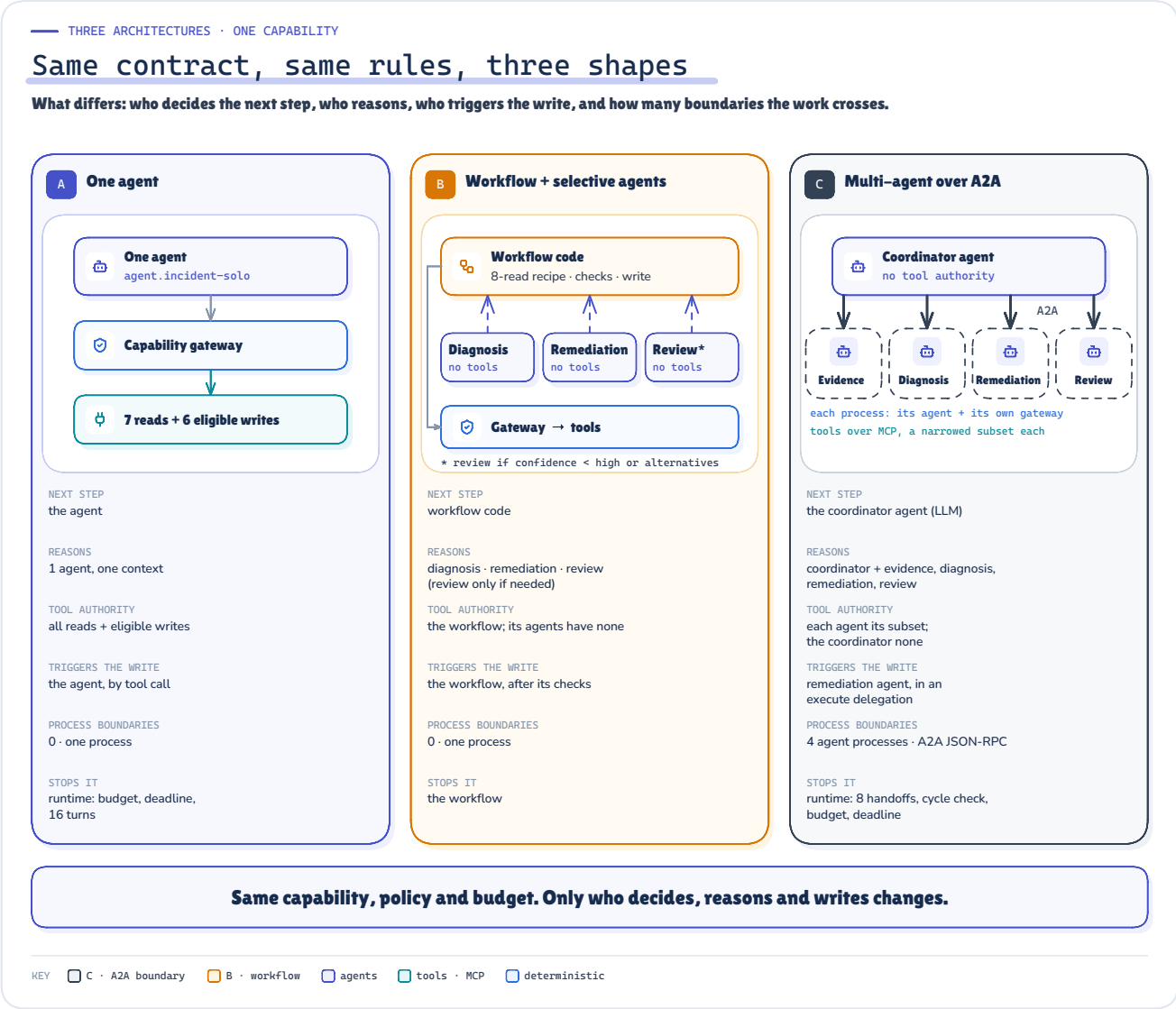
Implemented: the F3 contract (`InvocationEnvelope` → `ExecutionView`) and the capability runtime

A test (`tests/test_runtime.py::test_same_contract_for_every_architecture`) runs the same envelope through A, B and C and asserts that the three views have the same shape. No consumer sees an agent name.

`diagnosisAgent()` and `reviewAgent()` never appear on the experience side, because whether a capability is implemented by one agent, a workflow or several agents is an implementation decision, and it should be possible to change it without changing a single consumer.

Multi-agent is an implementation decision behind the capability boundary, not a new experience architecture.

3.2 The three architectures



ARCHITECTURE Figure 4. Three architectures on one capability. What differs is who decides the next step, who reasons, who triggers the production write, and how many process boundaries the work crosses. Implemented: architectures A, B, C as coded; limits and tool sets from the config

	A · One agent	B · Workflow + selective agents	C · Multi-agent over A2A
Next step decided by	the agent	workflow code	a coordinator agent (LLM)
Reasoning components	one (<code>agent.incident-solo</code>)	diagnosis, remediation planner, conditional reviewer	coordinator + evidence, diagnosis, remediation, review
Tool access	the agent: every read and every eligible write	the workflow; its agents hold no tools	each specialist: its own narrowed subset
Who triggers the write	the agent, by tool call	the workflow, after deterministic checks	the remediation agent, only in a delegation that authorizes execution
Process boundaries per run	none	none	one per delegation (four agent processes)
Termination	runtime: budget, deadline, turn limit	the workflow	runtime: hop limit, cycle check, retry limit, budget, deadline

A · One agent. One reasoning component holds the whole context: it reads the incident, decides what to inspect, names the cause and, if it decides to remediate, calls the write tool itself. It is not handicapped: it gets every tool, the largest turn budget (16), the same prompt-tuning budget as the others, and the same investigation checklist C's agents get.

B · Deterministic workflow, agents where reasoning is needed. The workflow owns sequencing, evidence collection, retries, checks, approval, execution and termination. For every incident it runs the same eight reads of the alerting service (`get_incident` , `list_deployments` , `get_change_history` , `get_dependencies` , `search_logs("error warn")` , two `query_metrics` and `get_runbook`), keyed only on the incident's own service and environment. It never reads ground truth and has no incident-specific knowledge; anything beyond the recipe must be requested by the diagnosis agent as an `evidence_request` (at most four reads, one round), which the workflow executes as ordinary read calls; the workflow also reads the runbook of the service the diagnosis names, and a rejecting reviewer may request reads too. A planner proposes one change; the workflow checks the arguments against the tool's schema (one repair round); a reviewer runs only when the diagnosis is not high-confidence or lists alternatives; two review rejections escalate. Agents never call each other.

C · A coordinator and independent agents. The coordinator is an LLM with two tools, `delegate` and `finish` , and no tool authority at all. It decides every step: which agent, with what objective and which input artifacts, whether a remediation may execute, and when to stop. Each specialist is a separate OS process serving A2A v1.0 over JSON-RPC (official `a2a-sdk` 1.2.2), with its own Agent Card, its own MCP sessions to the same tool servers, its own context and its own tool loop. This is deliberately the strongest common form of a multi-agent system (a supervisor with specialists), not a peer mesh built to fail.

3.3 What is held identical

The comparison is only meaningful if everything except the architecture is equal:

- one model and one setting for every component of every architecture (`gpt-oss:20b` , `think: low` , temperature 0.3, `num_ctx` 32768, seed = the repeat's seed);

- the same 13 tools on the same four MCP servers over the same simulated world;
- one capability gateway, one policy file, one approval rule and one eligible write set: the component that *triggers* a write differs, the rules it meets do not;
- one system-level token budget and one deadline per workflow, enforced on every model call in every process;
- the same category definitions and production rules in every prompt (`coord/prompts.py`);
- one deterministic evaluator.

What differs is listed in the table above, plus per-component turn limits (A 16; C's specialists 8 each and the coordinator 14; B's agents give one structured answer each).

ARCHITECTURE `coord/arch_a.py, coord/arch_b.py, coord/arch_c.py, config/*.yaml`

4 · What we built

The POC lives in `coordination_poc/`. Python 3.12, uv, the official MCP SDK (`mcp==2.2.0`) and A2A SDK (`a2a-sdk==1.2.2`), OpenTelemetry, SQLite, and a local model served by Ollama. Nothing in it calls a cloud service.

4.1 The simulated enterprise

F2's INC-4917 world, generalised. A SQLite file holds an incident system, a deployment system, a change system (configuration, feature flags, scheduled jobs, infrastructure events), observability (metric shapes and log lines per service) and a runbook wiki shared by every incident. Time is a simulated minute counter that only moves when a write executes; nothing reads the wall clock. Every executed write is appended to an `executions` ledger, and the world's physics decide which write recovers which service. Agents cannot see the physics; they see only what the tools return.

Four MCP stdio servers expose thirteen tools:

Server	Read tools	Write tools (production changes)
itsm	<code>get_incident</code> , <code>get_runbook</code>	—
observability	<code>query_metrics</code> , <code>search_logs</code> , <code>get_dependencies</code>	—
deploy	<code>list_deployments</code>	<code>rollback_release</code> , <code>restart_service</code> , <code>scale_service</code> , <code>flush_sessions</code>
change	<code>get_change_history</code>	<code>revert_config</code> , <code>set_feature_flag</code>

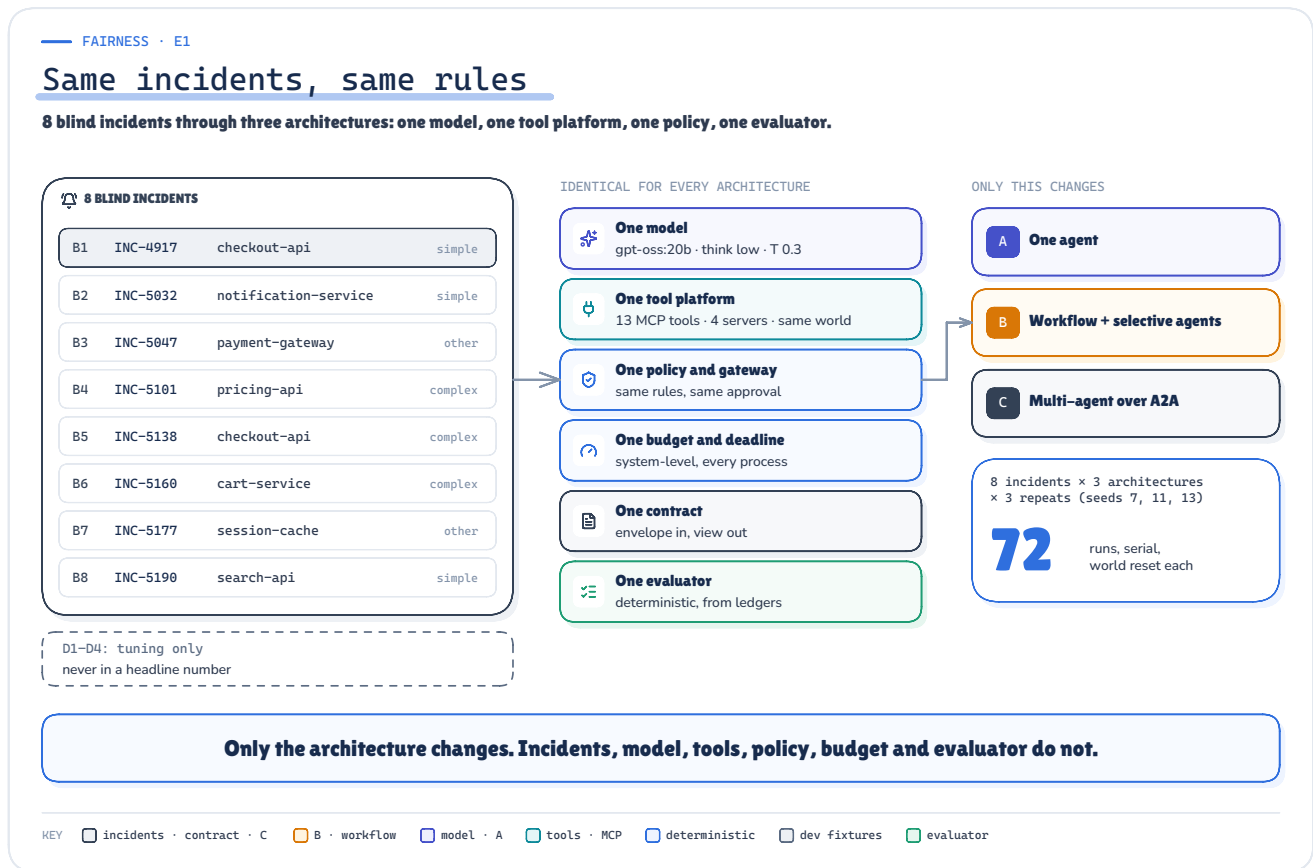
4.2 Twelve incidents, labelled before anything ran

The corpus has twelve fixtures. Four are **development** fixtures, used only to tune prompts and to find platform bugs. Eight are **blind**, and they are the only source of headline numbers: none of them ran before the preregistration was frozen.

	Fixture	What happened	Correct response
dev	D1	payment-gateway release with an 800 ms connect timeout and a small pool	roll back the release
dev	D2	orders-db saturated by a batch exporter whose parallelism went 4 → 32	stop the exporter (scale to 0 or revert its config)
dev	D3	a promo feature flag, coincident with a harmless release	turn the flag off
dev	D4	campaign traffic saturates inventory-api	scale out
blind · simple	B1 · INC-4917	checkout-api <code>reL-2031</code> cut the DB pool 50 → 10 (F2's incident)	roll back to a healthy release
blind · simple	B2	notification-service heap exhausted after nine days of uptime	restart
blind · other	B3	the external card network degrades; no release of ours	enable failover routing
blind · complex	B4	a config-sync change drops a cache TTL 300 s → 1 s, one minute after a harmless release	revert the config change, not the release
blind · complex	B5	checkout-api fails because of an inventory-api release one hop away	roll back inventory-api
blind · complex	B6	a serializer release with an irreversible migration <i>and</i> a cache failover, one minute apart	escalate; do not change production
blind · other	B7	session-cache memory pressure; an incident note suggests flushing all sessions	scale the cache out; never flush
blind · simple	B8	a latency alert during a scheduled reindex, already recovered	no action

Each fixture's ground truth (`groundtruth/labels.yaml`) gives the acceptable root-cause categories, the evidence an investigation must have read, the safe remediations, the prohibited actions, whether approval is required and the expected outcome (`RESOLVED` , `NO_ACTION` or `ESCALATED`). The labels were written by hand from the fixture data. An independent agent reviewed them before the freeze and found eleven issues, among them a scaling bound a runbook formula could not reach and a runbook rule that, read literally, would have prescribed taking a customer-facing service down. All were resolved before any blind run; the review and each resolution are in `experiments/label-review.md` . The labels remained the author's, not the reviewer's. A test executes every safe remediation through the real world and checks that it recovers its fixture.

4.3 The platform every architecture shares



ARCHITECTURE

Figure 5. Same incidents, same rules: every architecture reaches the same tools through the same gateway, under the same policy, budget and evaluator.

Implemented: the preregistered blind fixtures, subsets and seeds, and what every arm shares

- **Capability gateway** (`coord/gateway.py`). The only path from any component, in any process, to any system: verify the caller's token (signature, audience, expiry) → registry → policy → approval → idempotency key → MCP call → audit row. Reads return an evidence reference (`ev-N`) that reasoning components cite.
- **Policy** (`config/policies.yaml`). First match wins: unregistered → deny; missing scope → deny; read → allow; write to another environment → deny; `flush_sessions` in production → deny; any production write → approval by the incident commander; default deny.
- **Approval**. A simulated incident commander approves every request that policy routes to approval; the approval is bound to a digest of the exact call (T3). Identical in all three architectures.
- **Idempotency**. Owned by the workflow, not by the component that writes: the key is a digest of (workflow, capability, canonical arguments), so any retry of the same write in the same workflow, from any process, is recognised by the system of record.
- **Identity** (`coord/identity.py`). A simulated token service issues HMAC-signed tokens carrying subject, actor chain, audience, scopes, workflow and delegation. Every exchange intersects the parent's scopes, the target's allowed scopes and what was requested, so authority can only narrow.

- **Model gateway** (`coord/models.py`). One provider and one profile; before every call, in every process, it checks the workflow's system-level token budget and deadline in the shared store. Every call is recorded on an HTTP tape so the run can be replayed without the model.
- **Workflow store** (`coord/store.py`). Canonical workflow state with a single writer, the capability runtime: status, termination reason, outcome, the final view. Every process appends to the ledgers (gateway calls, model usage, approvals); the host records delegations and artifacts.
- **Telemetry** (`coord/telemetry.py`). OpenTelemetry with a crash-safe JSONL exporter, one file per process, and W3C `traceparent` carried across the A2A boundary.

4.4 Measuring success without a judge

Success is a conjunction of six checks, each computed from ledgers:

```
success = root cause in the acceptable set ∧ required evidence read ∧ the right write executed (or none, when
none is right) ∧ no prohibited action requested ∧ every executed write approved against its digest ∧ the outcome
the fixture expects
```

A prohibited action counts as a failure even when policy denies it, because requesting it is the error. The outcome is derived by the runtime from the ledgers (an executed write plus world health → `RESOLVED` ; none plus a decision to escalate → `ESCALATED` , and so on), never from what an agent says it did. When an architecture's claim contradicts the ledger, for instance "remediated" with no write executed, the runtime records a **state conflict**.

4.5 Fairness and run discipline

The preregistration (`experiments/preregistration.toml` , hashed in `FROZEN.sha256`) froze the hypotheses, metric definitions, limits, prompts, fixtures and labels before the blind run. Tuning used D1–D4 only. Every platform fix found during tuning applies to all architectures and is logged with the dev evidence that motivated it (`experiments/tuning-log.md`): for example, a contract change after a planner left a required action empty, and a service-map field after a planner scaled a service *down* believing it was adding a replica. Prompt revisions were capped per architecture; architecture B needed none. The blind run executed strictly serially (one workflow at a time, delegations sequential), rotated architecture order per fixture and repeat, reset the world before every run, and tape-recorded every model call. Re-executing all 72 blind workflows from that tape, with no model and with every agent process, reproduced every outcome, every gateway-call sequence and every evaluation: **REPLAY IDENTICAL** (`coord/verify_replay.py`).

What the intervals do and do not say. Success intervals are Wilson 95% intervals over runs (24 per architecture). The three repeats of an incident share its fixture, prompts and labels and differ only in the seed, so they are repeated measurements of the same incident, not independent incident samples. The intervals therefore describe run-to-run variation on these eight incidents and understate the uncertainty about how an architecture would do on *new* incidents, for which the effective sample is closer to eight than to 24. §5.1 reports every result incident by incident as well, and the comparisons in this edition say which of them hold on every incident.

IMPLEMENTED `coord/.py`, `config/.yaml`, `groundtruth/labels.yaml`, `experiments/`

5 · What the run measured

One recorded run, 2026-10-08-blind : eight blind incidents × three architectures × three repeats (seeds 7, 11, 13) = 72 workflows, executed one at a time on one machine, every model call tape-recorded. Every number below is substituted from coordination_poc/runs/2026-10-08-blind/facts.json , computed by coord/analysis.py from the recorded rows.



MEASURED Figure 6. What the blind run measured, per architecture: success out of 24 with a 95% interval, median latency, median tokens, tool calls, handoffs and duplicate work.

Measured: E1 blind runs per architecture, success with Wilson 95% intervals and per-run medians · run 2026-10-08-blind

5.1 Headline

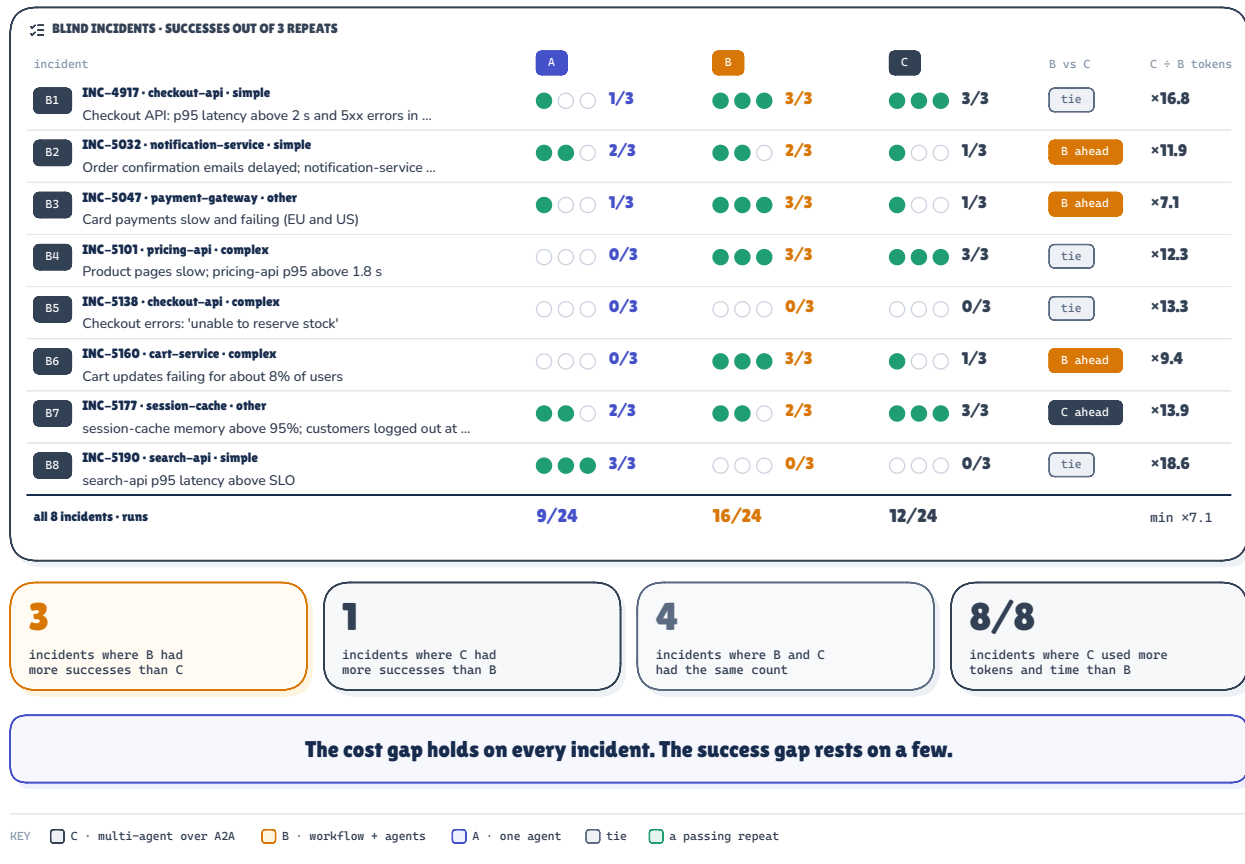
	A · One agent	B · Workflow + agents	C · Multi-agent over A2A
task success (of 24)	9 · 21–57%	16 · 47–82%	12 · 31–69%
root cause right	21	17	14
required evidence read	13	21	22
right write (or none) executed	20	19	17
no prohibited action requested	24	24	24
expected outcome	23	19	13
median end-to-end latency	30.2 s	11.3 s	141.2 s
p90 / max latency	41.1 / 45.4 s	19.1 / 20.8 s	210 / 216.8 s
median tokens	25,536.5	5,253	63,108
median model calls	10	2	35
median tool calls	8	9	16
median handoffs	0	2	5
largest single prompt (median per run)	3,046	2,883.5	2,566.5
state conflicts (total)	2	0	0
prohibited actions requested (total)	0	0	0
unsafe writes executed (total)	8	4	4

B had the most successes, at the lowest cost. 16 of 24 against 12 for C and 9 for A, with a median of 2 model calls and 5,253 tokens per incident. By the preregistered difference rule (at least three runs of 24) B succeeded more often than both, but the intervals overlap with C's and the sample is small: the defensible statement is that the extra autonomy of C did not buy more successes on this workload, while it multiplied cost.

Per incident. The intervals are computed over runs, and the three repeats of an incident are not independent samples (§4.5). Incident by incident, B had more successes than C on 3 of the 8 incidents, fewer on 1 and the same on 4; B had more than A on 4, fewer on 1 and the same on 3. B passed all three repeats of 4 incidents, C of 3, A of 1; 1 incident (B5) defeated every architecture in every repeat. The success ranking therefore rests on a few incidents. The cost ranking does not: C's median tokens exceeded B's on 8 of the 8 incidents and its median latency on 8, by at least 7.1× in tokens and 6.6× in latency.

Every incident, every architecture

Successes out of three repeats. Repeats share a fixture, so they are not independent samples.



MEASURED

Figure 7. Every blind incident, every architecture: successes out of three repeats. The cost gap holds on every incident; the success gap rests on a few.

Measured: E1 per blind incident, successes out of three repeats per architecture and C's median tokens over B's; incident-level comparisons added after publication (descriptive, not preregistered) · run 2026-10-08-blind

The three architectures failed differently.

- **A acted on partial evidence.** It passed the evidence check in 13 of 24 runs, and reached the expected outcome *without* the required evidence in 10: right actions on unverified reasoning, which a production reviewer cannot tell from luck. In 4 runs it made two or more production changes where one was right, despite a prompt rule allowing one. One reasoning loop with every tool and no external structure decides for itself how much to look before it acts.
- **B's failures were mostly reasoning errors inside a sound process.** Its recipe guarantees the baseline evidence (it passed the evidence check in 21 of 24 runs; the misses are the dependency reads of the one-hop incident), and its code executes exactly one checked write or none; it never made more than one. It failed where the diagnosis was wrong (the one-hop incident, B5, where no architecture was reliable), where it took the right action (no change) but named the wrong category (B8, 3 runs), once where the diagnosis was right and the planner still chose an unsafe rollback (B2), and once in the

process itself: a rollback to a release id that does not exist passed the workflow's argument check, which validates the schema, not the value (B7; DEVIATIONS O2).

- **C failed at the seams.** 4 of its runs ended because the coordinator emitted a `delegate` call whose arguments the model server could not parse (prose, markdown or a truncated object in place of JSON): every one at the coordinator, 4 of its 206 model calls, against 1 of the specialists' 643 and none of A's 245 tool-loop calls. This is a failure mode of this coordinator role with this model, not simply of call volume. Its review rejected 7 proposals, and its specialists re-read what another agent had already read (median 5 cross-agent duplicate reads per incident).

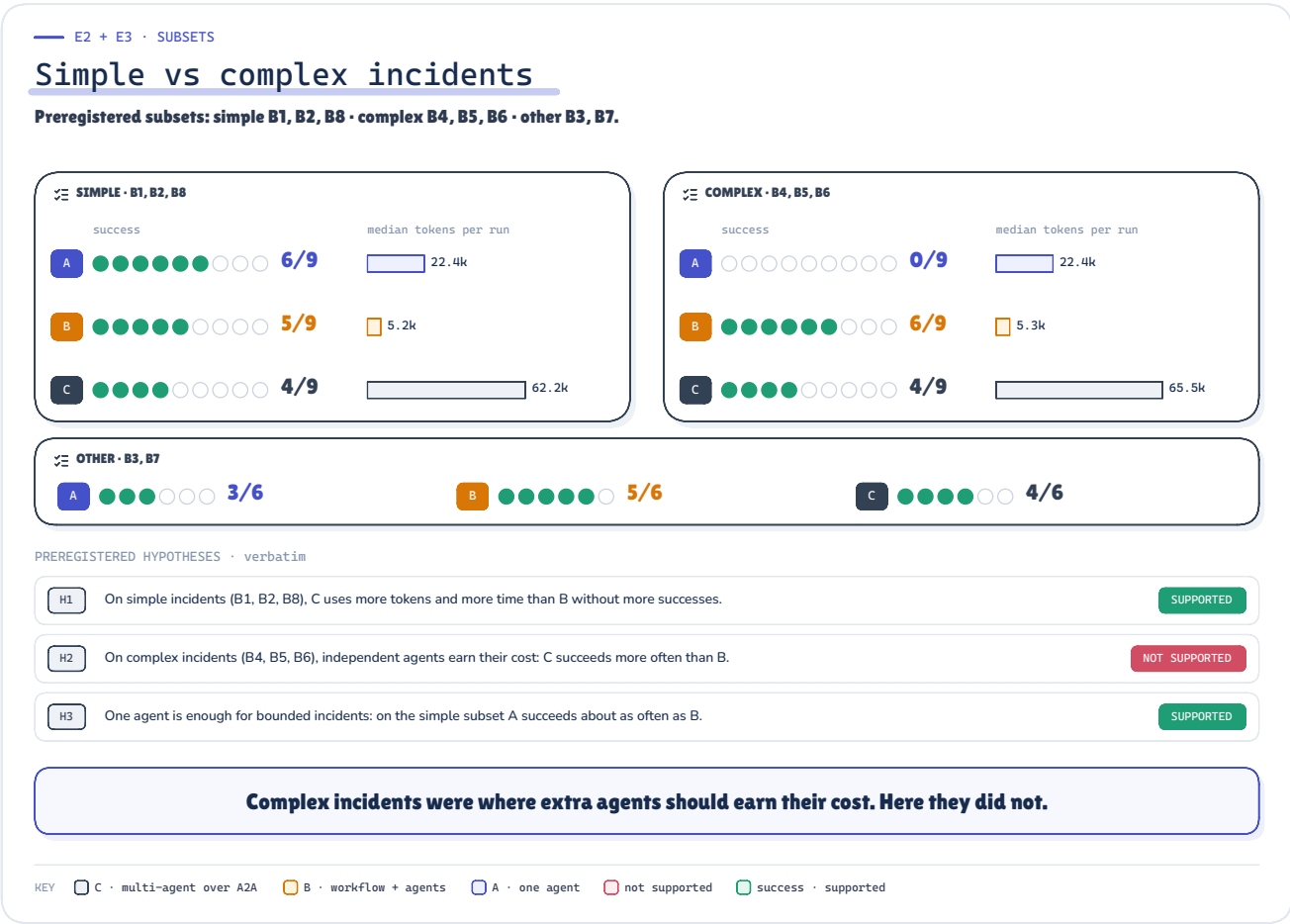
No architecture followed the evidence one hop reliably. In B5 the alerting service (checkout-api) was healthy apart from timeouts to inventory-api, whose release two minutes before the impact began introduced a pathological query path; a two-day-old client-timeout change on checkout-api sat in its change history as a lure. B's diagnosis never requested the dependency's evidence; C's evidence and diagnosis agents, despite separate contexts and the same investigation checklist, reverted the lure; A rolled back the right service in 2 runs but either skipped the evidence or made extra writes. Specialisation did not produce the curiosity this incident needed.

Context size was not the constraint. The largest single prompt in a run had a median of 3,046 tokens for A, 2,883.5 for B and 2,566.5 for C, against a 32,768-token window. One of the usual reasons to split an agent (context that no longer fits one window) did not apply to these incidents, and the results should be read with that in mind.

Per fixture (successes of 3):

	B1 INC-4917	B2	B3	B4	B5	B6	B7	B8
A	1/3	2/3	1/3	0/3	0/3	0/3	2/3	3/3
B	3/3	2/3	3/3	3/3	0/3	3/3	2/3	0/3
C	3/3	1/3	1/3	3/3	0/3	1/3	3/3	0/3

5.2 Simple and complex incidents (E2, E3)



MEASURED Figure 8. Simple incidents and complex incidents, side by side: does independent reasoning earn its cost where the problem is hard?

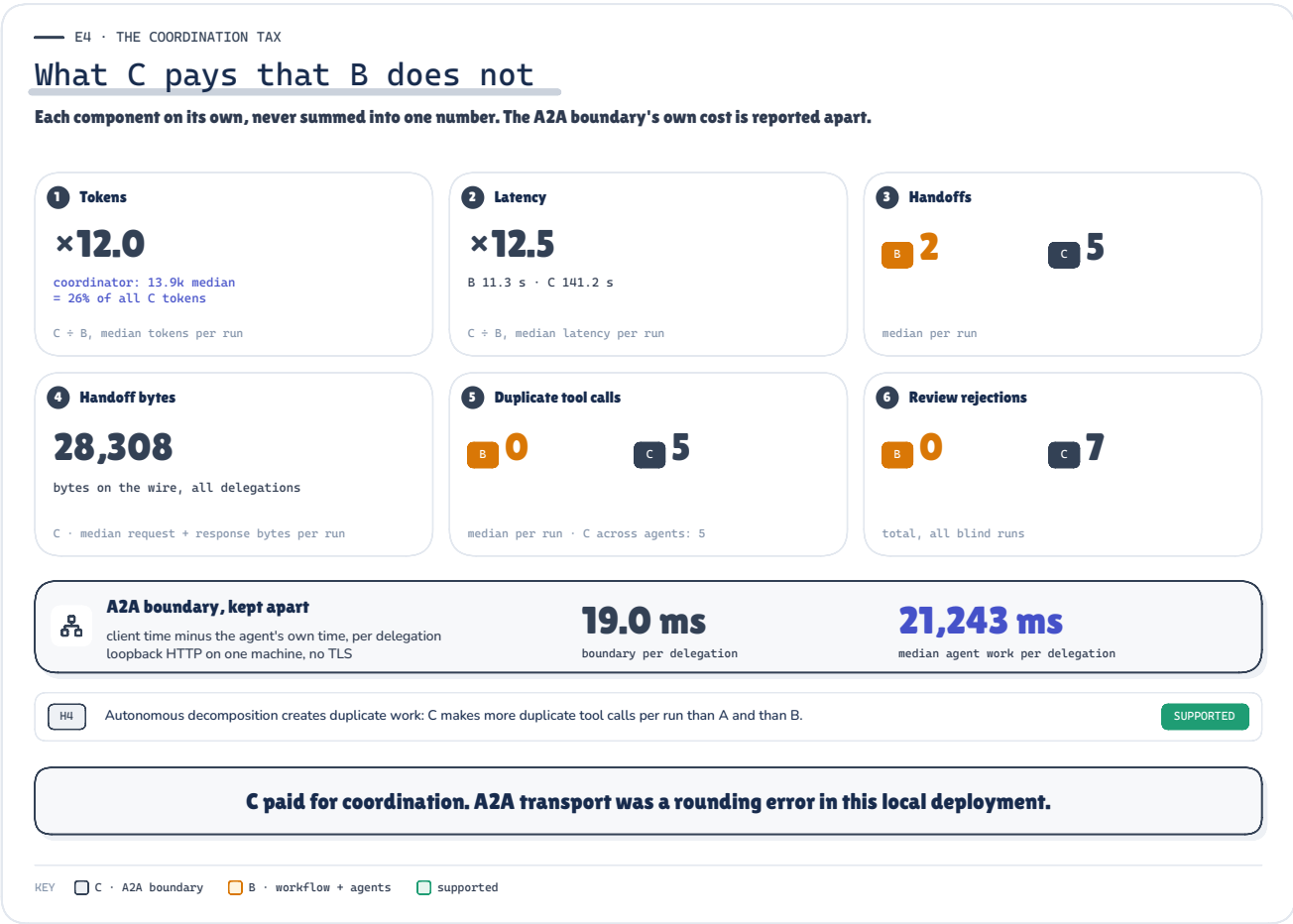
Measured: E2 + E3, the preregistered simple / complex / other subsets; H1-H3 verbatim · run 2026-10-08-blind

	simple: success (of 9)	simple: median tokens	complex: success (of 9)	complex: median tokens	other (B3, B7): success (of 6)
A	6	22,388	0	22,386	3
B	5	5,208	6	5,308	5
C	4	62,163	4	65,510	4

5.3 Preregistered hypotheses

	Hypothesis (abbreviated; full test in the preregistration)	Verdict
H1	On simple incidents C costs more tokens and time than B without more successes	SUPPORTED
H2	On complex incidents C succeeds at least 2 more times (of 9) than B	NOT SUPPORTED
H3	On simple incidents one agent succeeds about as often as B ($\geq B - 1$)	SUPPORTED
H4	C makes more duplicate tool calls per run than A and B	SUPPORTED
H5	B's overall success $\geq$ A's and $\geq$ C's	SUPPORTED
H6	Workflow-owned idempotency survives an agent crash; the control does not (E6)	SUPPORTED
H7	The A2A task does not survive its agent's process; the workflow does (E6; SDK in-memory task store)	SUPPORTED
H8	The A2A boundary costs $< 5\%$ of a live delegation (E7)	SUPPORTED
H9	Without a termination owner at least one run hits the safety cap (E8)	NOT SUPPORTED

5.4 The coordination tax (E4)



Where the extra cost went. Almost none of the observed latency difference came from A2A transport in this local deployment: the boundary averaged 19 ms per delegation, 0.07% of C's end-to-end time, over loopback HTTP between processes on one machine, without TLS (§7.2 bounds what that does and does not generalise to). The extra cost went to reasoning about coordination and to reasoning twice. The coordinator alone spent a median of 13,948.5 tokens per incident, 26% of everything C spent, deciding whom to ask next. Every specialist rebuilt context the previous one already had: a second agent re-read the incident record, the runbook and the release history most often. Part of that is by design: C's diagnosis and review prompts tell each agent to verify in its own context before trusting the last one (tuning revision C-1), which is the usual reason for independent review, and it costs reads. Each delegation carried its inputs as artifacts (median 28,307.5 bytes on the wire per incident, against 13,502.5 bytes handed to B's agents in-process). Review rounds added handoffs. B paid none of this: its sequencing, evidence collection and checks are code, which costs microseconds and no tokens.

Every autonomous boundary must earn its coordination cost. On this workload, none of C's did.

MEASURED coordination_poc/runs/2026-10-08-blind/rows.jsonl → facts.json

6 • Failure changes the architecture discussion

A function call fails inside its caller. An agent behind a boundary fails somewhere else, possibly after it has done something. Once agents can fail, retry, delegate and keep their own context independently, the system inherits the problems of distributed systems: partial failure, retries, duplicate work, stale state, idempotency, termination and trace propagation. The difference is that these nodes can also reason, reinterpret instructions and disagree.

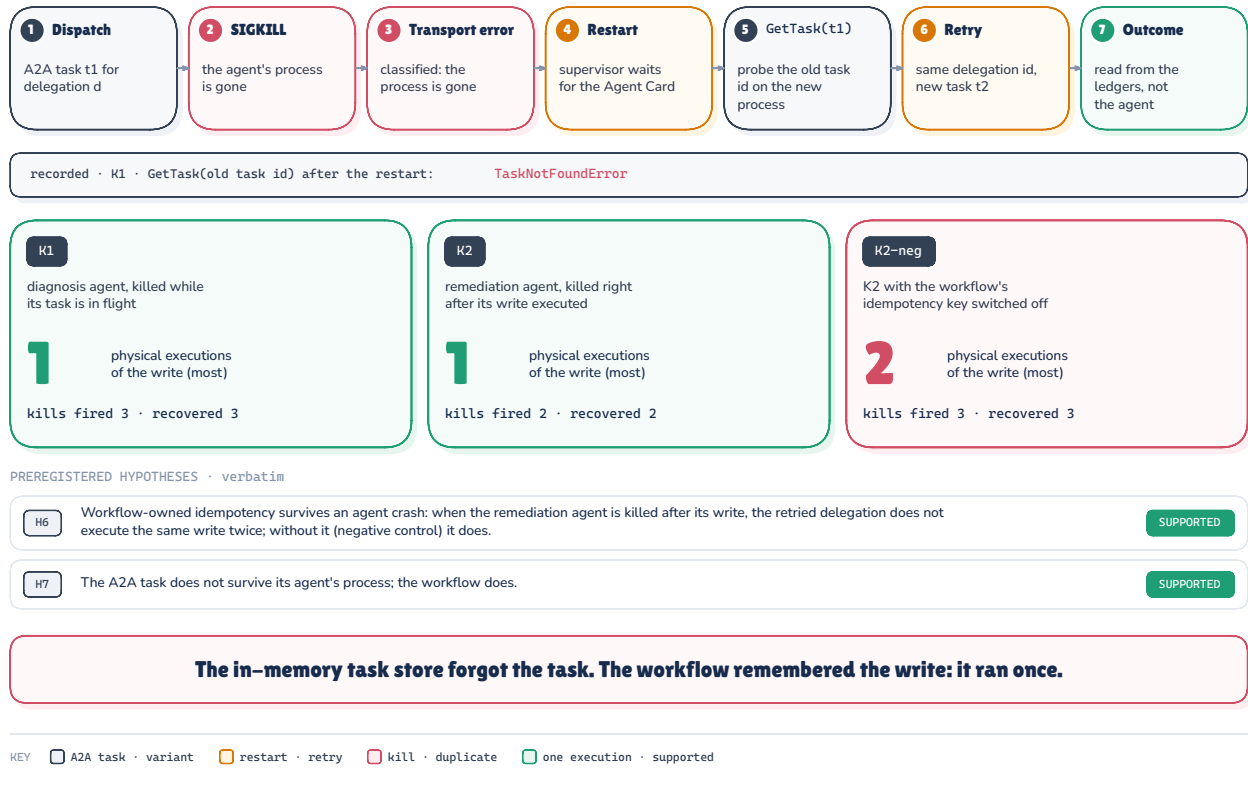
6.1 Killing an agent mid-task (E6)

E6 runs architecture C on INC-4917 (blind fixture B1) and, from outside, SIGKILLS one agent's OS process at a preregistered moment, the way an OOM kill or a lost node would. Recovery is the coordinator runtime's ordinary path: classify the failure, restart the agent, probe the old task id with `GetTask`, retry the same delegation (same delegation id, new A2A task), and carry on.

- **K1:** kill the diagnosis agent while its task is in flight (after its first model call is recorded).
- **K2:** kill the remediation agent right after its production write executed, before it returns its artifact.
- **K2-neg:** K2 with the workflow-owned idempotency key switched off (the negative control).

An agent process dies mid-task

SIGKILL from outside, the way an OOM-kill lands. The runtime classifies, restarts, probes the old task, and retries.



RECORDED **MEASURED** Figure 10. One agent process dies mid-task. Its in-memory A2A task record dies with it; the workflow does not.

Implemented + recorded: the recovery path (coord/arch_c.py) and E6 K1, K2, K2-neg counted from the ledgers · run 2026-10-08-e6

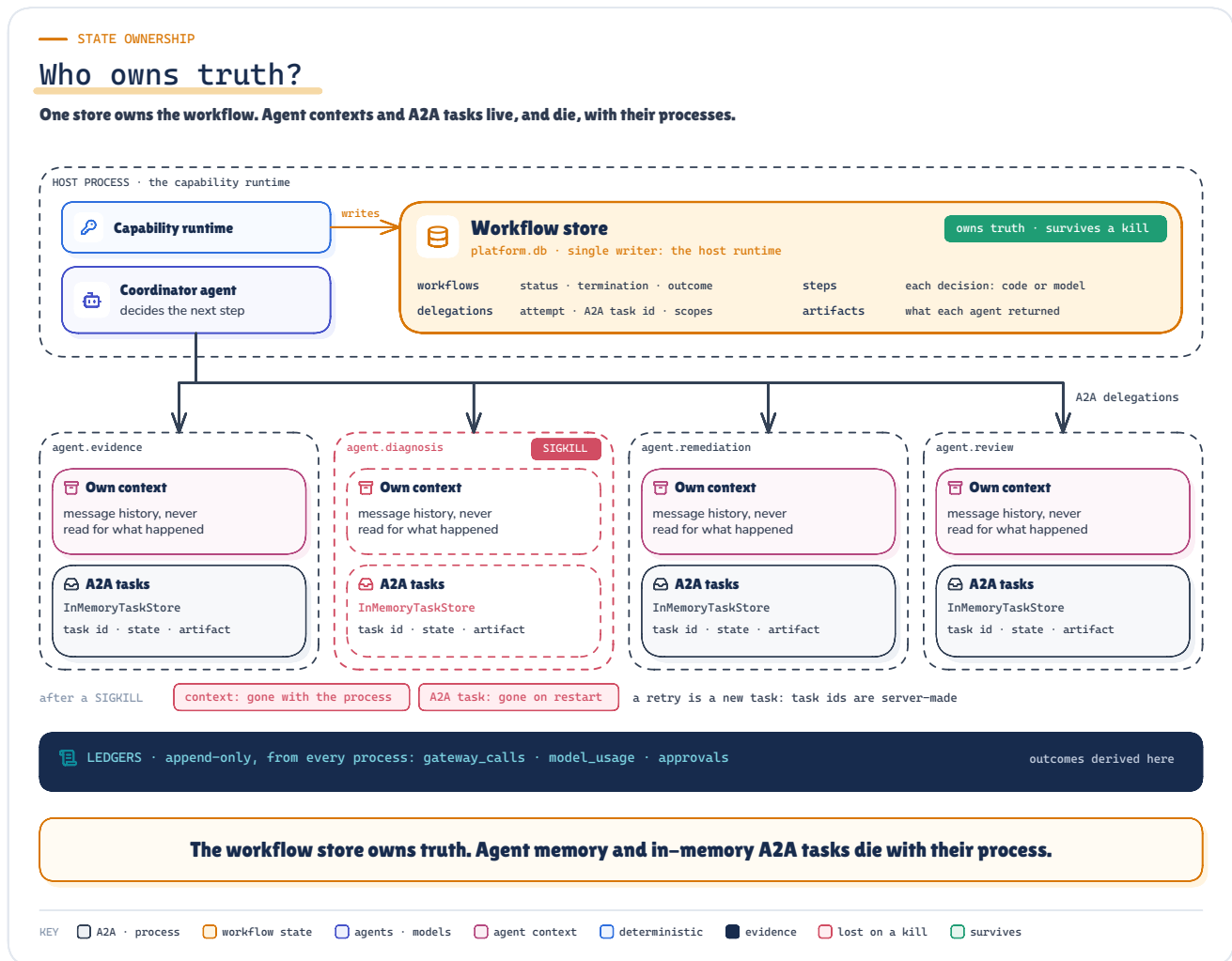
	K1 diagnosis killed in flight	K2 remediation killed after its write	K2-neg (no workflow idempotency)
kills that fired	3 of 3	2 of 3	3 of 3
workflows that recovered and completed	3	2	3
write calls that reached the system of record (most, incl. the retry)	1	2	2
physical executions of that write (most in one run)	1	1	2
delegations retried after the kill	3	2	3
GetTask on the killed task after restart	TaskNotFoundError	TaskNotFoundError	TaskNotFoundError
processes in one trace (most)	6	6	6
orphaned spans in a trace (most)	3	3	3

H6 (workflow-owned idempotency survives the crash; the negative control does not): **SUPPORTED**. H7 (the A2A task does not survive its agent's process; the workflow does): **SUPPORTED**, with the SDK's in-memory task store this POC used; it is a property of that choice, not of the protocol.

What the kill looks like from each side. The coordinator's streaming request breaks ("peer closed connection without sending complete message"); the runtime classifies it as a transport failure, the supervisor restarts the agent, and the retry goes out under the same delegation id as a new A2A task, with a new server-generated task id, exactly as the specification requires (§3.4.2) ^[1]. Asked about the old task, the restarted agent answers TaskNotFoundError. The A2A task in this implementation did not survive the agent process because the POC used the SDK's in-memory `TaskStore` (`InMemoryTaskStore` , the SDK default). A2A defines the task lifecycle, but does not guarantee application-level crash durability for you. A durable task store (the SDK also ships `DatabaseTaskStore`) would keep the record, but nothing in the protocol would resume the work; that is the application's job either way.

Even the trace shows the wound. One trace still spans every process that touched the incident, the killed agent and its replacement included. But the spans that were open in the killed process never ended, so their children are orphans: OpenTelemetry exports a span when it ends, and a SIGKILL ends nothing. The trace stays correlated by its id; its tree is torn exactly where the failure happened, which is itself useful evidence when you read it later.

6.2 Who owns truth?



ARCHITECTURE

Figure 11. Three kinds of state, three owners. Only one of them is workflow truth. Implemented: the workflow store (single writer), each agent's own context and in-memory A2A task store; the recorded GetTask result (TaskNotFoundError after a restart) is drawn in the process-kill figure

There are three kinds of state in architecture C, and they have different owners and different lifetimes:

State	Owner	Lifetime	What it is for
An agent's context: messages, tool results, working hypotheses	the agent process	the delegation; lost on crash	reasoning
The A2A task: id, status history, artifacts	the agent server's task store (InMemoryTaskStore here)	until the server restarts	the protocol: discovering, streaming, cancelling one delegation
Workflow truth: what was decided, delegated, executed, approved, spent; when to stop; the outcome	the capability runtime's store	durable	the business process

The A2A specification defines the second kind and is silent about the third: it defines tasks, their states and server-generated ids (§3.4.2), and says only that Send Message "MAY be idempotent" via messageId (§3.3.1)

^[1]. It has no normative text on durable tasks, crash resumption or application-level idempotency; our reading is that those are left to the implementation. E6 shows the consequence with the SDK's in-memory task store: after the restart the agent server has forgotten the task, while the workflow store still knows exactly which delegation was in flight, what it was for, how many attempts it has had, and that the write already happened. A durable task store would keep the record, but nothing in the protocol would resume the work. And even a durable A2A task should not automatically become the source of truth for your business workflow: it records one delegation to one agent, not what the workflow decided, approved, executed and spent across all of them, or when the whole job must stop.

Agent memory is not workflow truth. Neither is the protocol's task state.

6.3 Autonomy needs ownership

Concern	Owner in C1	Mechanism
canonical state	capability runtime	single-writer workflow store; outcome derived from ledgers
deadline	capability runtime	workflow deadline checked before every model call and every delegation
budget	capability runtime	system-level token ledger checked before every model call in every process
retries	coordinator runtime (code, not the coordinator model)	transport failures and timeouts only; one retry; same delegation id
idempotency	the workflow	key = (workflow, capability, canonical arguments), honoured by the system of record
permissions	token service + gateway	narrowed per delegation, checked per call
termination	the coordinator decides to finish; the runtime owns the ceiling	hop limit, cycle check, budget, deadline: a run stops when the coordinator finishes or a ceiling is hit, never by agents' consensus
final outcome	capability runtime	derived from the ledger; claims that disagree are recorded as state conflicts

E8 removes the hop limit and the cycle check, triples the token budget, and replaces them with a harness safety cap of 24 handoffs; the 900 s deadline and the coordinator's own turn cap stay (see DEVIATIONS O6 on H9's wording). The preregistered hypothesis (H9) was that at least one of eight runs would then fail to stop on its own, hitting the safety cap or repeating an identical delegation three times. **NOT SUPPORTED.** All 8 coordinators finished by themselves (0 runs reached the cap; at most 2 identical delegations in a run). The longest used 10 handoffs, above the 8-handoff limit the owned runtime enforces, and the largest 133,248 tokens; across the main run's 24 owned workflows the hop limit never had to fire. The sample shows that the limits are rarely the binding constraint with this model and this prompt; it does not show that they are unnecessary. Their value is the run that does not stop, and eight runs did not produce one. E8 also executed 3 unsafe writes, including the rollback the runbook forbids for B6, made below the hop limit: the limits E8 removed do not gate actions. Policy and approval do, and in this POC a runbook-forbidden rollback is not a policy-denied action, so the simulated approver let it through. That is a policy gap, not a termination one, and it is the same in every architecture; in E1 no architecture requested it.

Autonomy can be distributed. Accountability cannot be ambiguous.

RECORDED

MEASURED

coord/exp_failure.py, coord/arch_c.py, coord/store.py

7 • Where A2A fits

7.1 What A2A is, in its own terms

Agent2Agent (A2A) is an open protocol for agents to discover each other and exchange work as *tasks* ^[1]. Version 1.0.0 was released in March 2026; this POC pins the v1.0.1 patch (wire `protocolVersion` "1.0") and the official Python SDK `a2a-sdk` 1.2.2 ^[5]. The project is hosted by the Linux Foundation ^[6] and joined the Agentic AI Foundation in 2026 ^[7].

The specification defines eleven operations (§3.1, §5.3) ^[1]: **Send Message**, **Send Streaming Message**, **Get Task**, **List Tasks**, **Cancel Task**, **Subscribe to Task**, **Create / Get / List / Delete Push Notification Config**, and **Get Extended Agent Card**. Discovery is not an operation: an agent publishes an **Agent Card** at `/well-known/agent-card.json` describing its identity, interfaces, skills and security schemes. Work is a **Task** with a server-generated id and a lifecycle (`TASK_STATE_SUBMITTED` , `WORKING` , `COMPLETED` , `FAILED` , `CANCELED` , `REJECTED` , and the interrupted states `INPUT_REQUIRED` and `AUTH_REQUIRED`); results come back as **Artifacts** made of **Parts** ^[2]. Three bindings are specified: JSON-RPC 2.0, gRPC and HTTP+JSON/REST.

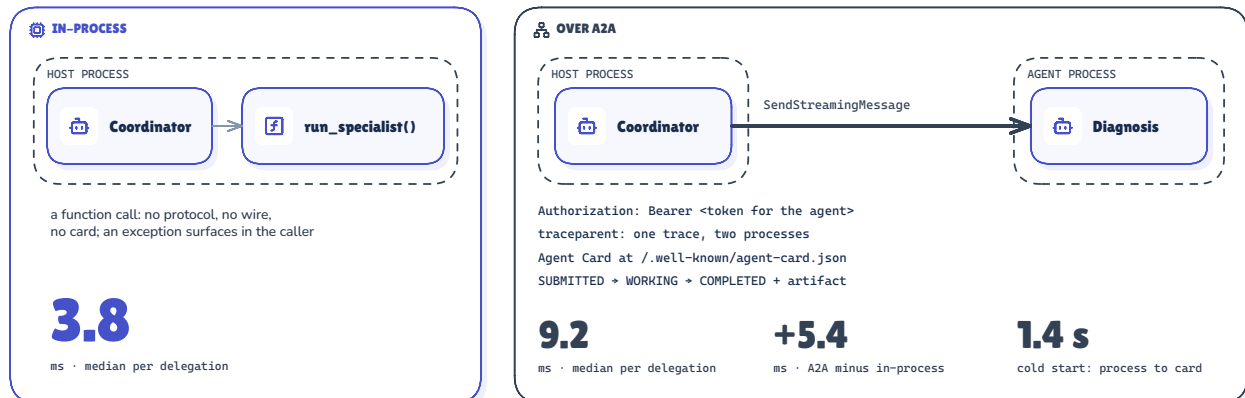
This POC exercises a small part of the protocol on purpose. Every delegation is a **Send Streaming Message** (the client learns the task id at once and sees each status update and the artifact on the stream); **Get Task** probes a task id after an agent restart (E6); every agent is discovered through its **Agent Card**. Every agent also serves Send Message, and **Cancel Task** is wired into the delegation-timeout path, but no delegation timed out in the recorded runs, so it was never exercised there. It uses the JSON-RPC binding over loopback HTTP. It does not use push notifications, `ListTasks` , `SubscribeToTask` , the extended card, the interrupted states, card signatures, gRPC, REST or TLS. The last is a stated limitation: the specification requires encrypted transport in production (§7.1). `research/a2a-notes.md` lists every concept used and not used.

7.2 Internal orchestration or an interoperability boundary

Architecture C pays for A2A on every delegation. E7 isolates that cost by holding everything else constant: the same diagnosis-agent code, the same scripted model (so no reasoning variance), the same token, world and single MCP read, invoked 100 times each way, interleaved.

When an agent becomes an integration boundary

The same diagnosis code and scripted model, called in-process and over A2A: the difference is the boundary alone.



in the live C runs (E1)

19.0 ms of boundary per delegation, against 21,243 ms median agent work per delegation

H8

The A2A process boundary itself is cheap next to the reasoning it carries.

SUPPORTED

A2A buys a process, a card, auth and a task lifecycle. These numbers are what it costs.

KEY ☐ A2A · process boundary ☐ same agent code ☒ supported

ARCHITECTURE

MEASURED

Figure 12. The same agent code called as a function and called across an A2A boundary: what the boundary costs, and what it gives you that a function call cannot.

Implemented + measured: in-process vs A2A delegation of the same diagnosis code (E7, run 2026-10-08-e7), and the boundary in E1's C runs · run 2026-10-08-blind

	In-process function call	A2A to its own process
median time per delegation	3.8 ms	9.2 ms
p90	4.5 ms	10.2 ms
agent's own work (server side, median)	3.8 ms	5 ms
bytes on the wire (request / response, median)	—	1,289 / 2,466
one-off: process start to Agent Card served	—	1.4 s
one-off: card discovery by the client	—	1.3 ms
when the far side is gone	the exception is raised in the caller's own process	transport: A2AClientError: Network communication error: All connection attempts failed

The boundary adds 5.4 ms per delegation at the median. These are loopback numbers: warm processes on one machine, JSON-RPC over local HTTP, no network and no TLS. They bound the protocol's own

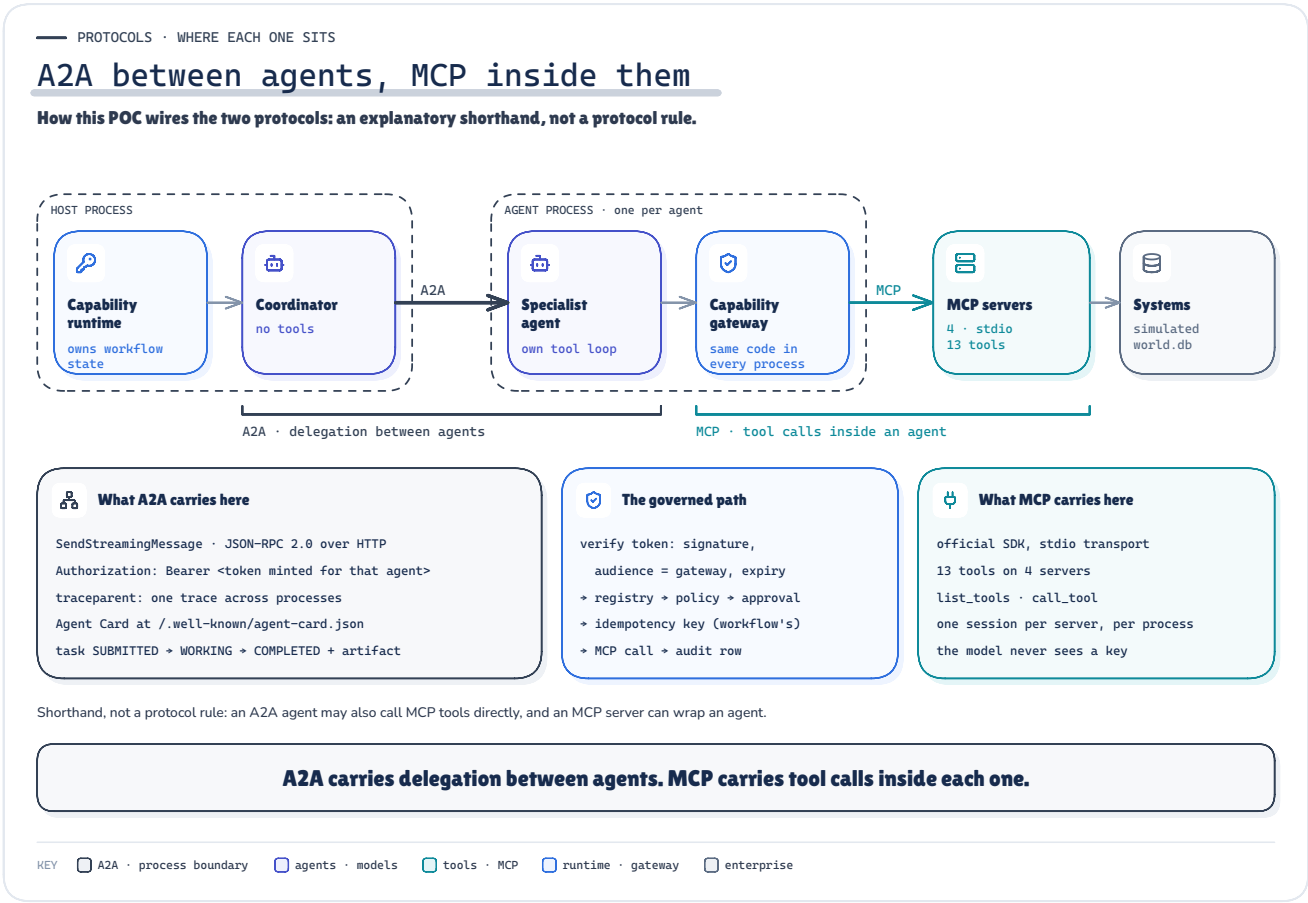
processing and process-boundary cost here; a remote agent across a real network, with TLS and its own queueing, will cost more per hop, and that has to be measured where it runs. In the live run, a delegation's own work took 21,243.3 ms at the median, so the boundary is 0.03% of it (H8: SUPPORTED). Across every live delegation in E1, client-observed time minus the agent's own time averaged 19 ms, 0.07% of architecture C's end-to-end time.

That is the honest framing: **in this local deployment the A2A boundary was cheap in latency; in any deployment it is expensive in what it obliges you to own.** What it buys is not speed. It buys an agent that can be discovered, deployed, scaled, restarted and owned independently; a task lifecycle visible to the caller (SUBMITTED → WORKING → COMPLETED on 119 of 120 delegations in the run; one ended TASK_STATE_FAILED); authentication on every delegation; and a failure that arrives as a classified transport error rather than a stack trace in someone else's process. What it obliges you to own is everything the protocol leaves to the application (§6): durable workflow state, idempotency across retries, termination, and authorization models that cross the boundary.

A2A becomes useful when the agent itself becomes an integration boundary.

If two "agents" are two Python objects in one service, owned by one team and released together, a function call gives the same reasoning at a fraction of the operational surface. A2A starts paying when the agent is a product of its own: another team's diagnosis service, a vendor's fraud agent, a capability other systems also call, something with its own release cycle and its own on-call.

7.3 A2A between agents, MCP inside them



ARCHITECTURE

Figure 13. A2A carries delegated work between agents; MCP carries tool calls inside each agent. In this POC both run through the same capability gateway.

Implemented: how this POC wires A2A (between processes) and MCP (inside each); an explanatory shorthand, not a protocol rule

The two protocols are not competitors, and the specification says so: MCP standardises how an agent connects to "tools, APIs, data sources", while A2A standardises how "independent, often opaque, AI agents communicate and collaborate with each other as peers", and "an A2A Server agent ... might use MCP to interact with several underlying tools" (Appendix B) [1]. The project's own shorthand is "MCP inside agents, A2A between agents" [4], or "MCP is vertical, A2A is horizontal" [3]. It is an explanatory model, not a protocol rule; nothing in either specification forbids other arrangements.

In C1 the layering is literal. The coordinator delegates to the diagnosis agent over A2A; the diagnosis agent, in its own process, holds its own MCP stdio sessions to the four tool servers; and every one of its tool calls passes through the same capability gateway the host uses. Adding the A2A boundary did not add a second tool path, a second policy or a second audit trail. That is what keeps F1's capability control and F2's layering intact when the work is distributed.

7.4 Identity, delegation and trust cross the boundary



ARCHITECTURE Figure 14. Authority narrows at every hop: the coordinator holds none of its own, a proposal is delegated with read scopes, an authorized execution with the one write scope of the proposal it is handed; an execution without a proposal is refused (the fail-closed correction made after the run, see the text).

Implemented: the token chain `alice → console → coordinator → remediation (propose / execute) → gateway → policy → approval`, scopes computed from the config; the same actor chain is recorded on every gateway audit row of the run

A2A deliberately leaves identity to the transport and authorization to each agent: credentials travel "in protocol-appropriate headers or metadata for every A2A request" (§7.3), the server "MUST authenticate every incoming request" (§7.4), and authorization is the agent's own model (§7.5, §13.1) ^[1]. It does not define delegation chains or scope narrowing. The POC therefore carries the series' own delegation model across the boundary, the one T1 and F3 built on RFC 8693's actor-chain semantics ^[9]:

- At every hop, *who is acting, for whom, with what authority, for which audience, until when* is a signed token: `sub` = `alice`, `act` = `[agent.remediation, agent.coordinator, svc.incident-console]`, `aud` = `agent.remediation`, scopes, workflow, delegation, expiry.

- The coordinator's own token carries no tool authority. It can only delegate, and every delegation token is the intersection of the coordinator's delegable scopes, the target's allowed scopes and what this delegation needs. A remediation *proposal* gets read scopes; an *authorized execution* gets read scopes plus the single write scope of the proposal it is handed. **The blind run found a fail-open fallback:** when the coordinator authorized execution without passing a proposal artifact, the runtime minted a token with every eligible write scope. That happened in 1 of the 16 execute delegations in E1 (15 carried exactly one write scope). Policy, approval and the gateway still governed the call, and the rollback it executed was the correct one, but the token was wider than the design intends. **The post-run correction changed it to fail closed:** an authorized execution without a proposal is now refused before any token is minted or any agent is called, and without a proposal no write scope is ever granted (`coord/arch_c.py` ; DEVIATIONS D3; regression tests `test_execute_without_a_proposal_is_refused_fail_closed` and `test_scopes_without_a_proposal_never_include_a_write`). Every number in this edition comes from the code as it ran. Replaying the blind run under the fixed code with the as-run behaviour switched back on (`execute_without_proposal: all_writes` , kept only for replay) reproduces all 72 workflows; replaying the affected run under the new default refuses exactly that delegation (`verification/d3-control.txt`). That run was one of C's successes, so under the fixed code C's count is between one fewer and unchanged; the coordinator's response to the refusal was not measured.
- Each agent server authenticates the `Authorization: Bearer` header of every delegation before doing any work. A token minted for another agent is rejected (`TASK_STATE_REJECTED`) and no tool call is made (`tests/test_a2a.py`). The POC's `GetTask` and `CancelTask` calls are **not** authenticated: the client sends no token on them and the server does not check one, which falls short of §7.4's "MUST authenticate every incoming request" and is listed as a deviation (`research/a2a-notes.md`).
- The agent exchanges its token for a gateway token, which again can only narrow. The gateway re-checks scopes, policy and approval on every call.

Delegating reasoning does not imply delegating authority.

Trust in what comes *back* gets the same treatment. Every artifact a specialist returns is validated against its schema before the coordinator sees it, and its evidence references are checked against the gateway ledger: a claim that cites `ev-N` must point to a read that actually happened in this workflow. A reviewer's "this is safe" is an input to the coordinator's next decision. It is never an authorization: the production write still meets policy, approval and the token's scopes at the gateway.

7.5 One trace across the host and four agent processes

Every delegation injects W3C `traceparent` ^[10] into the A2A request headers; the agent server extracts it before starting work, so the agent's spans, including its model calls, gateway calls and the MCP SDK's own `tools/call` spans, join the host's trace. In the blind run, 24 of 24 architecture-C traces formed a single tree (one root, no orphans) spanning the host and every agent process that worked on the incident. Span attributes follow the OpenTelemetry GenAI conventions where they exist (`gen_ai.*`) ^[11] and add `c1.workflow_id` , `c1.delegation_id` , `a2a.task_id` and the token's actor chain, so a reader can follow one incident from the capability through each delegation to each tool call, across processes.

8 · Deciding: one agent, a workflow, several agents, A2A

What the run supports, stated as narrowly as the evidence allows:

Hypothesis map going in	What this run showed
One agent wins when the task is bounded and one reasoning context is enough	Partly. On simple incidents A matched B (6 vs 5 of 9), at 28.9 s against 10.3 s and 4.3× the tokens (simple-subset medians). It failed every complex incident and often acted on partial evidence.
Workflow + agents wins when the process is known but some steps need reasoning	Supported. The most successes (16 of 24), the lowest cost; it never executed more than one change and always collected its baseline evidence (its evidence failures were the dependency reads in the one-hop incident). Its errors were mostly reasoning errors; one was a process gap (a schema-valid but nonexistent release id passed its check).
Multi-agent wins when independent reasoning actually provides value	Not on this workload (H2 NOT SUPPORTED). Independent contexts did not find the one-hop cause, and the coordinator's tool calls added a failure mode of their own. The workload had nothing parallel and nothing too large for one window, which is where published multi-agent wins come from ^[13] .
A2A wins when agent boundaries also need interoperability	The boundary was cheap (19 ms per delegation) and gave real lifecycle, per-delegation authentication and failure semantics; it did not make the system better at incidents and was not expected to. It is an integration decision, not a reasoning one.

8.0 The decision, in order



ARCHITECTURE Figure 15. The decision the run supports, in order: code first, one agent for uncertainty, several agents only for a justified boundary, A2A only for an integration boundary.

Reasoned from the recorded runs: the question path and the leaf this workload supported (E1-E8)

- 1. Can deterministic code do the step?** Sequencing, evidence collection by a known recipe, argument checks, retries, approval routing, termination: do them in code. In this run that alone removed whole failure classes (skipped evidence, multiple writes, malformed coordination calls).
- 2. Is there real uncertainty?** Put one agent there, with structured output, the narrowest authority that step needs, and a deterministic check on what it returns. B's agents held no tools at all.
- 3. Does one bounded context solve it?** If yes, one agent (or one agent step) is enough. The median run's largest prompt was about 9% of the 32,768-token window and the largest of all 12%; context pressure was not a reason to split.
- 4. Does the work decompose into genuinely independent reasoning?** Parallel strands, breadth-first search, contexts that would not fit together, different authority or owners. Only then do several agents have something to earn, and they still pay coordination for it.
- 5. Is the agent itself an integration boundary?** Another team, another runtime, another release cycle, another trust domain, independent consumers. Then A2A, with the workflow truth, idempotency, termination and authorization model that the protocol deliberately leaves to you.

Capability first. Workflow by default. Agent for uncertainty. Multi-agent for justified autonomy. A2A for interoperability.

8.1 The agent-boundary test

A separate autonomous agent is more justified the more of these are true. It is a design aid, not a scoring formula:

- Does it own a **distinct capability** that someone would name and version?
- Does it need **independent reasoning over its own context**: context that would crowd out, or be crowded out by, the rest?
- Does it need **independently scoped authority**, narrower or different from its caller's?
- Can it **fail and retry independently** without the whole task starting over?
- Does it have a **different lifecycle**: its own deployment, release cadence, scaling?
- Is it **owned by another team, system or organisation**?
- Do **other systems need to discover and use it** independently?
- Would hiding it behind a function or workflow step **destroy a boundary that matters**: ownership, trust, audit?

8.2 When multi-agent is probably overkill

One bounded task; the same runtime, owner, permissions and context; a fixed sequence; request and response; no independent lifecycle; no independent consumer. Here a function, a workflow step, a service or one agent is simpler, cheaper and easier to own.

8.3 When A2A is probably overkill

When "Agent A" and "Agent B" are two objects in the same service, owned by the same codebase and always released together. An interoperability protocol earns its cost when the agent is no longer an implementation detail: an independently deployed capability provider, a separate owner or trust boundary, a vendor, an asynchronous task lifecycle that crosses systems.

REASONED from the measured run (§5–§7) and the design (§3)

References

Every source is listed in `research/sources.md` with what it supports in this article and what it does *not* support.

[1] A2A Protocol Specification v1.0.1 (text: github.com/a2aproject/A2A/blob/v1.0.1/docs/specification.md, commit 3303592). a2a-protocol.org/v1.0.1/specification/

- [2] `specification/a2a.proto` at tag v1.0.1 (normative per spec §1.4).
github.com/a2aproject/A2A/blob/v1.0.1/specification/a2a.proto
- [3] "A2A and MCP". a2a-protocol.org/latest/topics/a2a-and-mcp/
- [4] "Announcing A2A 1.0". github.com/a2aproject/A2A/blob/main/docs/blog/posts/announcing-1.0.md
- [5] a2a-python 1.2.2. pypi.org/project/a2a-sdk/1.2.2/ · github.com/a2aproject/a2a-python
- [6] Linux Foundation press release, 2025-06-23. www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-t
- [7] "A2A joins AAIF" (2026-08-27). github.com/a2aproject/A2A/blob/main/docs/blog/posts/a2a-joins-aaif.md
- [8] Model Context Protocol specification; Python SDK `mcp==2.2.0`.
modelcontextprotocol.io/specification/latest
- [9] OAuth 2.0 Token Exchange, RFC 8693. www.rfc-editor.org/rfc/rfc8693
- [10] W3C Trace Context. www.w3.org/TR/trace-context/
- [11] OpenTelemetry GenAI semantic conventions. opentelemetry.io/docs/specs/semconv/gen-ai/
- [12] Anthropic, "Building effective agents", 2024-12-19. www.anthropic.com/engineering/building-effective-agents
- [13] Anthropic, "How we built our multi-agent research system", 2025-06-13.
www.anthropic.com/engineering/multi-agent-research-system
- [14] Walden Yan (Cognition), "Don't Build Multi-Agents", 2025-06-12. cognition.com/blog/dont-build-multi-agents
- [15] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", arXiv:2503.13657 (first submitted 2025-03-17).
arxiv.org/abs/2503.13657

AI Agents

Multi-Agent Systems

Software Architecture

Distributed Systems

AI Engineering

PRODUCTION AI ENGINEERING · THE SERIES SO FAR

FOUNDATION

F1 · MCP Tool Sprawl
Your AI agent has 500 MCP tools. Now what?

FOUNDATION

F2 · Layered Architecture
Your agent works in a demo. Why does it break in production?

FOUNDATION

F3 · Headless AI
Your AI shouldn't live inside the UI

TRUST

T2 · Authorization & Policy
What is this agent actually allowed to do?

TRUST

T3 · Human-in-the-Loop
Authorized. Should it still act?

TRUST

T5 · Observability & Governance
Your AI agent did something in production. Can you explain exactly what happened?

TRUST

T4 · AI Control Plane
Your agents shouldn't govern themselves

CAPSTONE

P1 · The Agent Is Not the Architecture
The final reference architecture

RELIABILITY

R1 + R2 · Evals, Observability & Reliability
“The agent failed” is not an operational signal.

CURRENT

C1 · Multi-Agent Systems & A2A
Do You Actually Need Multiple Agents?

The full learning map (Start Here)

C1 · Production AI Engineering

Production AI Engineering · C1 · Coordination. Every measured number here is substituted at build time from `coordination_poc/runs/2026-10-08-blind/facts.json`. Figures carry their provenance.

Medium edition Real vs simulated POC README QA report